

Here is the complete, granular architectural specification for each microservice in your retirement planning cluster based on your product design document.

1. Income & Inflow Service

A. Data to Be Captured / Data Sources

- **Job Inflows:** Base Salary (Monthly), Bonus (Annual), RSUs/Stock Options (Quarterly), Signing Bonus (One-time).
- **Rental Properties:** Long-term and Short-term variables including Monthly Rent/Income and Maintenance/Operating Expenses.
- **Tax Metadata:** Tax Configuration Flag (SYSTEM_CALCULATED vs. MANUAL_OVERRIDE) and Manual Effective Tax Rate value.

B. Calculations to Be Performed

- **Net Property Cash Flow:**
$$\\$\\text{Net Rental Income} = \\text{Gross Rent/Income} - \\text{Expenses}\\$\\$$$
- **Gross Annual Baseline:** Sum of all job components normalized to a 12-month calendar year + net rental cash flows.
- **Net Annual Take-Home:**
$$\\$\\text{Net Income} = \\text{Gross Income} \\times (1 - \\text{Effective Tax Rate})\\$\\$$$

C. Output Payload Schema (IncomeState)

```
"user_id" "usr_98765"  
"summary"  
  "total_gross_annual" 245000.00  
  "total_net_annual" 183750.00  
  "effective_tax_rate" 0.25
```

```
"monthly_cash_flow_grid"  
  "month" 1 "gross" 18500.00 "net" 13875.00  
  "month" 4 "gross" 26500.00 "net" 19875.00
```

2. Asset Ledger Service

A. Data to Be Captured / Data Sources

- **Asset Catalog:** Inputs for Income-Generating assets (Cash, Bonds, Equities, Annuities, 401K, IRA, Roth, College Fund, Emergency Fund, Goal-Specific Funds, CDs, Private Equity, Properties) and Non-Income Generating assets (Primary Home, Custom categories).
- **Performance Benchmarks:** Current Balance, 2025 ROI, Average 10-Year Historic Return, and user-adjustable Target Return.
- **Transaction Logs:** Real-time inflows/outflows, targeted future goal allocations, asset sales records (and destination of proceeds), and dividend/interest treatment preferences (Cash Income vs. Reinvested).

B. Calculations to Be Performed

- **Portfolio Weighing:**
$$\\%\\text{ of Total Portfolio}_{\\text{Asset}} = \\frac{\\text{Current Balance}_{\\text{Asset}}}{\\sum \\text{Balances}_{\\text{All Assets}}} \\times 100\\$\\$$$

- **Real-Time Valuation:**

$$\text{Real-Time Value} = \text{Historical Value} - \text{Total Outflows} + \text{Total Inflows}$$

- **Weighted Return Rate:**

$$\text{Average Portfolio Return} = \sum (\% \text{ of Total Portfolio}_{\text{Asset}} \times \text{Target Return}_{\text{Asset}})$$

C. Output Payload Schema (AssetLedgerState)

```
"user_id" "usr_98765"
"net_worth_contribution"
"total_asset_value" 1150000.00
"average_portfolio_target_return" 0.068
```

```
"assets"
```

```
"id" "ast_441"
"type" "401K"
"category" "Income_Generating"
"balance" 450000.00
"portfolio_percentage" 39.13
"historic_roi_2025" 0.08
"target_return_next_10y" 0.07
"reinvest_dividends" true
```

3. Debt & Liability Service

A. Data to Be Captured / Data Sources

- **Liability Registry:** Accounts grouped by type (Mortgage, Credit Card, Student Loans, Custom fields).
- **Terms & Amortization:** Remaining Principal Balance, End-of-Term/Target Payoff Dates, Interest Rates, Early/Late Fee Penalties, and Scheduled Monthly Payment Amounts.
- **Credit Card Treatment Switch:** Configuration defining whether credit card debt balances are funneled into Monthly Recurring Expenses.

B. Calculations to Be Performed

- **Aggregate Liability Debt:** Total outstanding principal sums across all open accounts.
- **Amortization Tracker:** Decrementing principal balances monthly by checking actual payments against accruing interest.

C. Output Payload Schema (DebtState)

```
"user_id" "usr_98765"
"total_debt_balance" 385000.00
"total_monthly_debt_service" 2450.00
"debts"
```

```
"type" "Mortgage"
"remaining_balance" 375000.00
"interest_rate" 0.045
"monthly_payment" 1900.00
"target_payoff_date" "2045-06-01"
```

```
"type" "Credit_Card"
"remaining_balance" 100000.00
"interest_rate" 0.19
"monthly_payment" 550.00
"treat_as_recurring_expense" true
```

4. Goals & Contribution Service

A. Data to Be Captured / Data Sources

- **Goal Configuration Entities:** Goal Type (Lifestyle, Legacy/Gifting, Capital Expenditures), Target Dollar Amount, Target Duration, and Target Priority.
- **System Inflow Feed:** Surplus cash metrics supplied by the Income Service.
- **Monthly Tracker Inputs:** User validation checks confirming if monthly planned savings milestones were achieved or missed.

B. Calculations to Be Performed

- **Waterfall Logic Engine:** Allocates cash down the priority matrix:
 - *Step A:* Prioritize CASH_SAVINGS until Emergency Fund targets are fully funded.
 - *Step B:* Prioritize accounts targeting high-interest debt (>7% APR).
 - *Steps C–F:* Max out employer-matched 401(k) allocations, fund Roth accounts, then satisfy major lifestyle/custom goals.
- **Variance Auditing:** Tracking the actual vs. planned target path:
$$\text{Goal Tracker Deviation} = \text{Planned Savings} - \text{Actual Input}$$

C. Output Payload Schema (GoalsRecapState)

```
"user_id" "usr_98765"
"income_saved_percentage" 18.5
"waterfall_status"
  "emergency_fund" "status" "FULLY_FUNDED" "current" 25000 "target" 25000
  "toxic_debt" "status" "ACTIVE_PAYDOWN" "current" 5000 "target" 0
```

```
"recap"
```

```
"goal_name" "Kids College Education"
"target_amount" 100000.00
"projected_end_state" 85000.00
"status" "UNDER_FUNDED"
"gap" 15000.00
```

5. Net Worth Orchestration Service

A. Data to Be Captured / Data Sources

- **Event Consumers:** Subscribes to real-time asynchronous messages broadcast from the Asset Ledger Service and Debt Service.

B. Calculations to Be Performed

- **Net Worth Equation:**
$$\text{Net Worth} = \text{Total Assets Value} - \text{Total Liabilities Debt}$$

C. Output Payload Schema (NetWorthState)

```
"user_id" "usr_98765"  
"timestamp" "2026-06-02T13:18:00Z"  
"net_worth" 765000.00  
"metrics"  
  "gross_assets" 1150000.00  
  "gross_debt" 385000.00
```

6. Simulation & Scenario Analysis Service

A. Data to Be Captured / Data Sources

- **Planning Parameters:** Current Age, Target Retirement Age, Desired Country/Region to Retire, Retirement-Specific Goals, and anticipated structural Lifestyle Expense Changes.
- **Macro Environment Overrides:** Long-term Inflation Rate assumptions, Cost-of-Living adjustment parameters (as a percentage of baseline US costs), and Portfolio Return benchmarks.

B. Calculations to Be Performed

- **Target Corpus Formula:** Calculates the aggregate capital pool required at retirement age by adjusting current expenses for inflation over the years, calculating returns via portfolio target rates, and scaling the total by the target country's Cost-of-Living index.
- **Required Contribution Model:** Solves for the required yearly contribution step-ups needed to successfully bridge the gap between current tracked asset positions and the calculated target retirement requirement.

C. Output Payload Schema (RetirementScenarioMatrix)

```
"user_id" "usr_98765"  
"simulation_metadata"  
  "target_savings_required_at_retirement" 2150000.00  
  "projected_savings_at_target_age" 1850000.00  
  "funding_gap" 300000.00  
  
"remediation"  
  "action_required" "INCREASE_SAVINGS"  
  "additional_annual_contribution_required" 14200.00
```

```
"interactive_curve"  
  "age" 35 "projected_assets" 150000.00 "required_trajectory" 150000.00  
  "age" 65 "projected_assets" 1850000.00 "required_trajectory" 2150000.00
```

Service 7: Income & Inflow Service (Enhanced Capability)

Your original design laid out a static model for capturing base salaries, bonuses, and rental properties. This enhanced service connects real-world bank data directly to those targets.

DOCX

A. Data to Be Captured / Data Sources

- **Expected Inflow Baselines:** The static user inputs already defined (Job income, bonuses, long/short-term rental targets).

DOCX

- **Actual Inflow Feed:** Live transaction webhooks (Plaid/Yodlee) filtering for depository account direct deposits, ACH transfers, and check clearing.

B. Calculations to Be Performed

- **Income Variance Matrix:**
- **Irregular Income Normalization:** A rolling 3-month or 12-month smoothing filter for RSUs, annual bonuses, or short-term rental seasonality to calculate an accurate, non-volatile monthly baseline for the *Goals Service* to use.

DOCX

C. Output Payload Schema (`EnhancedIncomeState`)

JSON

```
{  
  "user_id": "usr_98765",  
  "cycle_period": "2026-06",  
  "expected_gross_monthly": 15000.00,  
  "actual_gross_received": 14200.00,  
  "variance_status": "DEFICIT",  
  "variance_amount": -800.00,  
  "rental_property_health": [  
    { "property_id": "prop_101", "expected_net": 1200.00, "actual_net":  
1150.00 }  
  ]  
}
```

Service 8: Dynamic Expense & Budget Service (New Service)

Objective: Automatically ingest, categorize, and track daily consumer spending against living expenses and discretionary targets. This provides real-time data to determine the user's available surplus cash for savings goals.

DOCX

A. Data to Be Captured / Data Sources

- **Historical Transaction Streams:** Financial institution data feeds capturing merchant names, ISO category codes, transaction dates, and settlement amounts.
- **Budgetary Threshold Objects:** User-defined or auto-generated limits split into three core categories:
 - *Fixed/Essential Expenses:* Mortgage/Rent payments, utility patterns, student loan minimums, and insurance premiums.

DOCX

- *Discretionary Expenses:* Dining out, entertainment, shopping, and travel.
- *Sinking Funds / Irregular Expenses:* Semi-annual car insurance, holiday gifts, or annual subscriptions.

B. Calculations to Be Performed

- **Auto-Categorization Rules Matrix:** A rule-matching system that filters transactions into their structural spending types.
- **The Available Surplus Runway:** Calculates the true remaining money available for saving on a rolling monthly basis:
- **Rolling Average Burn Rate:** A calculation that determines average daily spending trends to project whether the user will overspend their discretionary budget before the month ends.

C. Output Payload Schema (`SpendingLedgerState`)

JSON

```
{
  "user_id": "usr_98765",
  "budget_month": "2026-06",
  "spending_summary": {
    "fixed_committed": 4200.00,
    "discretionary_spent_to_date": 1150.00,
    "discretionary_remaining_allowance": 850.00,
    "projected_month_end_surplus": 1350.00
  },
  "burn_rate": {
    "average_daily_spend": 57.50,
    "pacing_status": "ON_TRACK"
  }
}
```

```

    },
    "top_categories": [
      { "category": "Groceries", "allocated": 600.00, "spent": 240.00 },
      { "category": "Dining_Out", "allocated": 400.00, "spent": 310.00 }
    ]
  }
}

```

3. How This Connects to the Rest of the Cluster

Adding spending management changes your entire application from a passive planning tool to an active financial loop:

1. **Feeding the Goals Waterfall:** The *Dynamic Expense & Budget Service* calculates the `projected_month_end_surplus`. This calculated surplus dollar figure is published as an event. The *Goals Service* consumes this event and updates its **Waterfall Allocation Engine**, allowing it to notify the user exactly how much extra money they can route to their Emergency Fund or 401(k) this month.

DOCX

2. **Updating the Simulation Engine:** If the *Expense Service* detects that a user's baseline fixed expenses have structurally increased over a 6-month period (e.g., due to inflation or lifestyle changes), it emits a `FixedLifestyleExpensesIncreased` event. The *Simulation Service* picks this up and automatically runs a fresh background projection to see if the user's retirement target age needs to adjust.

DOCX

1. Transaction Event Message Formats (Kafka Schema Definitions)

To enable asynchronous communication between the **Dynamic Expense & Budget Service** and the **Goals & Contribution Service**, we establish highly structured JSON message payloads. These events are broadcast to a central message broker whenever financial parameters shift.

DOCX

Event A: `SurplusCashCalculatedEvent`

- **Publisher:** Dynamic Expense & Budget Service
- **Consumer:** Goals & Contribution Service
- **Trigger:** Fired automatically at the end of a billing cycle or upon a manual budget reconciliation step.

JSON

```

{
  "event_id": "evt_01H4X7B8Z9M2P5R1QW349001",
  "event_type": "SurplusCashCalculatedEvent",

```

```

"timestamp": "2026-06-02T20:25:14Z",
"payload": {
  "user_id": "usr_98765",
  "billing_period": "2026-05",
  "total_actual_income": 14200.00,
  "total_actual_fixed_expenses": 4200.00,
  "total_actual_discretionary_spending": 4350.00,
  "calculated_surplus_cash": 5650.00
}
}

```

Event B: BudgetDriftAlertEvent

- **Publisher:** Dynamic Expense & Budget Service
- **Consumer:** Goals & Contribution Service, Simulation Service
- **Trigger:** Fired if actual spending pattern tracking deviates significantly from the user's target model.

JSON

```

{
  "event_id": "evt_01H4X7B8Z9M2P5R1QW349002",
  "event_type": "BudgetDriftAlertEvent",
  "timestamp": "2026-06-02T20:25:14Z",
  "payload": {
    "user_id": "usr_98765",
    "analysis_month": "2026-06",
    "category_affected": "Discretionary_Total",
    "baseline_allocation": 2000.00,
    "projected_month_end_spend": 2650.00,
    "drift_percentage": 32.5,
    "impact_severity": "MEDIUM",
    "action_hint": "DEGRADE_ASPIRATIONAL_GOALS"
  }
}

```

2. Budget vs. Actual Dashboard UI (Wireframe Architecture)

The frontend layout must present complex transaction data cleanly alongside long-term strategic retirement pathways. This interface unifies current cash-flow reality with future goal projections.

DOCX+ 1

```

+-----+
+-----+
| [Logo] PLATFORM COCKPIT      Dashboard  Ledgers  Goals  Scenarios
(User: Alex V.) |
+-----+
+-----+
|
|
| +-----+
+-----+ |
| | CURRENT MONTH AT A GLANCE (June 2026)
| |

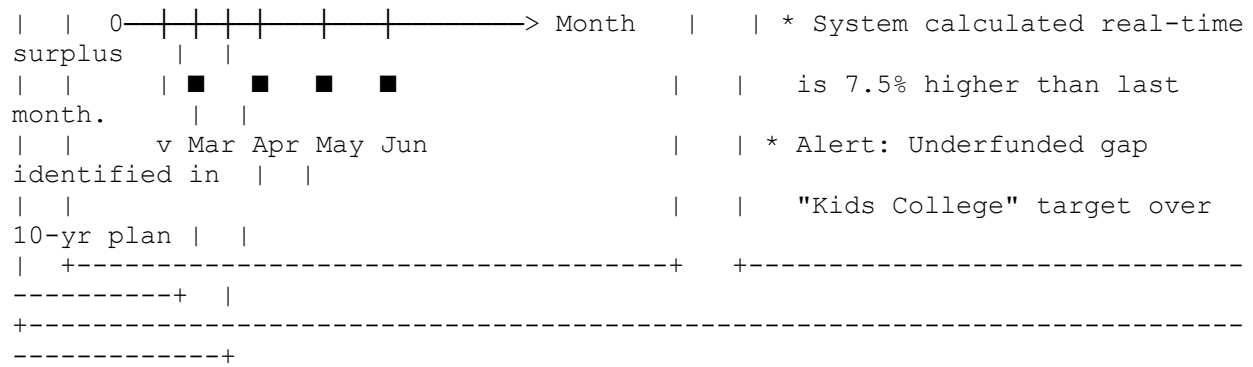
```



```

| | |
| | | [ Income Received ] [ Fixed Expenses ] [ Discretionary
Spend ] | |
| | $14,200 Actual $4,200 Committed $1,150 / $2,000
Cap | |
| | ($15,000 Target) (On Track) (Pacing: Stable)
| |
| | +-----+ +-----+ +-----+
-----+ | |
| | | Progress: [|||||]--] | | Progress: [|||||] | | Progress:
[|||||] | | |
| | +-----+ +-----+ +-----+
-----+ | |
| +-----+
-----+ |
|
|
| +-----+ +-----+
-----+ |
| | LIVE TRANSACTION CATEGORIES | | GOAL WATERFALL SEQUENCER
| |
| |
| | Groceries: $240 / $600 [|||---] | | Current Est. Monthly Surplus:
$8,850 | |
| | Dining Out: $310 / $400 [|||||] | |
| |
| | Utilities: $180 / $200 [||||-] | | Priority Target Execution
Flow: | |
| | Travel: $420 / $800 [|||---] | |
| |
| | | 1. Emergency Safety Net [
FULFILLED ] | |
| | [ View All Clear Transactions ] | | 2. Toxic Debt (>7% APR) [
FULFILLED ] | |
| +-----+ | 3. Employer 401(k) Match [
ACTIVE ] | |
| | | | Routing $1,250 of
monthly surplus | |
| +-----+ | 4. Roth IRA Contribution [
PENDING ] | |
| | MONTHLY CASH FLOW VARIANCE HISTORY | |
| |
| |
| | | [ Recalibrate My Funding
Priority ] | |
| | Net Surplus | +-----+
-----+ |
| | ^ |
| |
| | | |
-----+ | +-----+
| | | | BACK-END NOTIFICATION ALERTS
| |
| | | |
| |

```



3. Execution Pipeline Logic for the Dashboard

1. **State Synchronization:** The user drops a new transaction or adjusts a category allowance budget slider on the client interface.
2. **Stateless Request Processing:** The frontend fires a stateless `PUT` request to the API Gateway, which redirects the payload safely into the **Dynamic Expense & Budget Service**.
3. **Surplus Recalculation:** The Expense service saves the record, checks the state database, and runs the `Surplus Available` formula against the user profile.
4. **Asynchronous Event Fanout:** The calculated surplus integer is dropped onto the message broker cluster.
5. **Downstream Read-Side Mutations:** The **Goals Service** updates the user's active waterfall allocation pipeline layout, while the **Simulation Service** recalculates long-term terminal assets behind the scenes[cite: 3]. The UI consumes the newly calculated read-model projections to change the frontend status gauges instantly.

DOCX

Data Table

To ensure data is captured exactly once, avoids field duplication, and remains synchronized across all microservices, the system uses a **Shared Core Data Contracts Pattern**. Each microservice owns its specific domain fields and exposes them to other services using globally unique IDs (user_id, account_id, goal_id) rather than duplicating data across tables.

1. Global Identity Shared Keys (Common Headers)

These keys exist in every table to link data across services without duplicating descriptive fields.

- user_id (**UUID**): Unique global identifier for the consumer.
- account_id (**UUID**): Unique reference key for an institutional link.
- entity_id (**UUID**): Unique identifier for a specific debt, asset, or goal.

2. Microservice Table Schemas (Zero Duplication)

Table 1: Asset Ledger Service (Asset Domain)

- **Domain Responsibility:** Owns absolute asset values and historical performance metrics. It does *not* track what the asset is being saved for.

Field Name	Data Type	Constraints / Enums	Description
asset_id	UUID	PRIMARY KEY	Unique identifier for the asset account.
user_id	UUID	FOREIGN KEY	References the global user profile.
institution_name	VARCHAR(100)	NOT NULL	Name of the holding institution (e.g., Vanguard, Chase).

Field Name	Data Type	Constraints / Enums	Description
tax_bucket	ENUM	PRE_TAX, ROTH, TAXABLE	Dictates future tax treatment during withdrawals.
asset_category	ENUM	INCOME_GENERATING, NON_INCOME_GENERATING	Classifies asset nature.
current_balance	DECIMAL(15, 2)	NOT NULL, ≥ 0	Real-time market value.
cost_basis	DECIMAL(15, 2)	DEFAULT 0.00, ≥ 0	Original purchase price (for capital gains math).
historic_roi_2025	DECIMAL(5, 4)	NULLABLE	The historical return rate achieved in 2025.
target_return_next_10y	DECIMAL(5, 4)	NOT NULL	User-adjustable performance benchmark.
reinvest_dividends	BOOLEAN	DEFAULT TRUE	True if dividends are buying more shares;

Field Name	Data Type	Constraints / Enums	Description
			False if cash income.
last_updated	TIMESTAMP	NOT NULL	Automated timestamp of the last balance sync.

Table 2: Income & Inflow Service (Income Domain)

- **Domain Responsibility:** Owns regular base inflows and property operations. It does *not* track asset valuations.

Field Name	Data Type	Constraints / Enums	Description
income_source_id	UUID	PRIMARY KEY	Unique identifier for the income stream.
user_id	UUID	FOREIGN KEY	References the global user profile.
income_type	ENUM	W2_JOB, LONG_TERM_RENTAL, SHORT_TERM_RENTAL, OTHER	Categorizes the incoming revenue stream.
gross_amount	DECIMAL(15, 2)	NOT NULL	Raw pre-tax amount received.
frequency	ENUM	ONETIME, WEEKLY, BIWEEKLY,	Interval at which the

Field Name	Data Type	Constraints / Enums	Description
		MONTHLY, QUARTERLY, ANNUAL	income hits.
operating_expenses	DECIMAL(15, 2)	DEFAULT 0.00	Costs deducted immediately (crucial for rental calculations).
use_manual_tax_override	BOOLEAN	DEFAULT FALSE	Flags if the engine should bypass automated tax brackets.
manual_effective_tax_rate	DECIMAL(5,4)	DEFAULT NULL, 0 to 1	Custom historical tax rate provided by the user.

Table 3: Debt & Liability Service (Debt Domain)

- **Domain Responsibility:** Owns lending parameters, balances, and specific payoff rules. It does *not* evaluate if the debt is an expense or a retirement goal.

Field Name	Data Type	Constraints / Enums	Description
debt_id	UUID	PRIMARY KEY	Unique identifier for the liability account.

Field Name	Data Type	Constraints / Enums	Description
user_id	UUID	FOREIGN KEY	References the global user profile.
debt_type	ENUM	MORTGAGE, CREDIT_CARD, STUDENT_LOAN, OTHER	Grouping type for debt paydown logic.
remaining_principal	DECIMAL(15, 2)	NOT NULL, \$≥ 0\$	Current outstanding principal balance.
interest_rate_apr	DECIMAL(5,4)	NOT NULL, \$≥ 0\$	Annual Percentage Rate used to flag toxic debt.
minimum_monthly_payment	DECIMAL(15, 2)	NOT NULL, \$≥ 0\$	Hard baseline cash required to service the debt monthly.
target_payoff_date	DATE	NULLABLE	End-of-term or user-targeted payoff constraint.
early_payment_fee	DECIMAL(15, 2)	DEFAULT 0.00	Penalty cost for

Field Name	Data Type	Constraints / Enums	Description
			paying off the principal early.
treat_cc_as_recurring_expense	BOOLEAN	DEFAULT FALSE	Switch to route minimum payment directly to the expense engine.

Table 4: Dynamic Expense & Budget Service (Spending Domain)

- **Domain Responsibility:** Tracks actual transactional consumer outlays. It completely avoids asset/debt parameters by compiling category totals.

Field Name	Data Type	Constraints / Enums	Description
budget_id	UUID	PRIMARY KEY	Unique identifier for the monthly budget instance.
user_id	UUID	FOREIGN KEY	References the global user profile.
billing_period	VARCHAR(7)	NOT NULL (Format: YYYY-MM)	The specific calendar month being tracked.
category_name	VARCHAR(50)	NOT NULL	Individual categories (e.g., Groceries, Dining_Out).

Field Name	Data Type	Constraints / Enums	Description
allocated_cap	DECIMAL(15,2)	NOT NULL, \$ \ge 0\$	The user-defined spending ceiling for the month.
actual_spent_to_date	DECIMAL(15,2)	DEFAULT 0.00, \$ \ge 0\$	Live sum of aggregated cleared transactions.

Table 5: Goals & Contribution Service (Goals Domain)

- **Domain Responsibility:** Controls the financial priority strategy and tracks milestone variances. Notice it does *not* hold account balances or income figures; it holds routing rules.

Field Name	Data Type	Constraints / Enums	Description
goal_id	UUID	PRIMARY KEY	Unique identifier for the saving milestone .
user_id	UUID	FOREIGN KEY	References the global user profile.
associated_target_id	UUID	NULLABLE (Ref: asset_id debt_id)	Links the goal straight to the asset or debt it modifies without

Field Name	Data Type	Constraints / Enums	Description
			duplicating details.
goal_taxonomy	ENUM	LIFESTYLE, LEGACY_GIFTING, CAPITAL_EXPENDITURE	Structural category governing inflation behavior.
target_amount	DECIMAL(15,2)	NOT NULL, \$≥ 0\$	The total financial dollar goal target.
target_duration_months	INT	NOT NULL, \$> 0\$	Timeline runway to complete the goal.
strategy_priority_tier	INT	1 to 6 (1 = Highest)	Placement index inside the automated waterfall loop.
planned_monthly_contribution	DECIMAL(15,2)	NOT NULL	Calculated milestone savings run-rate requirement.
actual_monthly_contribution_verified	DECIMAL(15,2)	DEFAULT 0.00	Logged monthly deposit confirmed

Field Name	Data Type	Constraints / Enums	Description
			by user check-ins.